Trippi — Team Task Plan & Coordination Doc

Goal: Ship the AI-powered trip planner from current state (auth + UI done) to a working demo. **Team:** 3 developers, 3 parallel branches. **Branch model:** Everyone branches from `main`, merges via Pull Request, no direct pushes to `main`.

How to Read This Doc

- 🟢 **Task** — independent work, you can start anytime
- 🔴 **BLOCKER** — you cannot start this until someone else delivers something
- ⚠️ **DECISION** — team must agree together before anyone proceeds
- 🔄 **SYNC POINT** — stop and check in with the team
- 🤝 **PAIR** — recommended to do together, not solo

Week 0 — Before Anyone Touches a Branch (Day 1, ~2 hours, ALL THREE TOGETHER)

These must happen before splitting up. Skipping this step is the #1 cause of merge hell later.

- ⚠️ **DECISION 0.1: Pick the LLM provider.** Anthropic Claude API, OpenAI, or Google Gemini. Decide based on (a) free credits available, (b) who has an account, (c) streaming support if Branch B wants it. Write the choice in this doc.
    - CLAUDE 3.5/4 SONNET for user interaction
    - Google Gemini 1.5/2.5 for Data Architecture
- ⚠️ **DECISION 0.2: Sync or async Agent 2?** Re-read the mentor's note. Recommended starting point: **synchronous Agent 2** for the first two weeks, with async as a stretch goal. Document the choice.
    - SYNC AGENT FIRST -> THEN ASYNC

⚠️ **DECISION 0.3: Define the AI orchestrator function signature.** Write the exact Python signature everyone will code against. Example:
def handle_user_message(prompt: str, session: ChatSession, user: User) -> str:
"""Returns the assistant's reply text. Side effect: may update UserPreference."""

- nu ne atingem de views.py, cream un orchestrator.py intr-un folder agents.

⚠️ **DECISION 0.4: Define the chat AJAX endpoint contract.** Example:
POST /chat/<session_id>/message/Body: { "prompt": "..." }Returns: { "session_id": int, "assistant_message": { "role": "assistant", "content": "...", "sent_at": "..." } }

⚠️ DECISION 0.4: Chat AJAX Endpoint Contract

URL: POST /chat/<session_id>/message/

REQUEST:
- Content-Type: application/json
- CSRF token required (Django standard): X-CSRFToken header or in body
- Body:
```
{
  "prompt": "string (user's message, required, non-empty)"
}
```

RESPONSE (success, HTTP 200):
```
{
  "session_id": int,
  "message": {
    "role": "assistant",
    "content": "string (the AI's reply)",
    "sent_at": "ISO 8601 timestamp (e.g., 2026-05-19T14:32:15Z)"
  },
  "error": null
}
```

RESPONSE (error, HTTP 400/500):
```
{
  "session_id": int or null,
  "message": null,
  "error": "string (human-readable error message)"
}
```

NOTES:
- Branch A (backend): implements views.py to parse JSON, call orchestrator, return JSON
- Branch B (frontend): sends JSON via fetch(), reads message.content, appends to DOM
- This endpoint is NEW; the form POST still exists as fallback for non-JS browsers
- session_id is always returned so the frontend can show it in the URL or for debugging
- sent_at is included so the frontend can display timestamps accurately

- 🤝 **TASK 0.5: Pre-refactor `trips/views.py`.** Together, create the `agents/` app: `python manage.py startapp agents`. Create `agents/orchestrator.py` with a stub `handle_user_message()` that just returns `"stub response: " + prompt`. Rewire `chat_view` to call it. Commit this to `main` so all three branches start from the same clean baseline. -am facut branch aici (pavel)
- 🟢 **TASK 0.6: Create branches.** `feature/ai-agents`, `feature/chat-ux`, `feature/user-preferences`. Each person checks out their own.
- 🟢 **TASK 0.7: Add `.env.example`** to the repo with all expected variables (DB credentials, AI API key placeholder). Helps everyone keep their local env in sync.

🔄 **SYNC POINT:** After Week 0 is done, all three of you should be able to `python manage.py runserver` from your branch and see the app work end-to-end with the stub reply. If anyone can't, fix that before proceeding.

BRANCH A — AI Agents (`feature/ai-agents`)

Owner: *[Teammate 1 name here]* **Estimated effort:** 50–70 hours (the longest branch). **Key files:** `agents/` (new app), `trips/views.py` (minimal touches), `.env`.

Week 1 — Agent 1 (Concierge), end to end

- 🟢 **A1.1: Set up the LLM client.** Create `agents/llm_client.py` — a thin wrapper around the chosen provider's SDK. Read API key from `.env`. Single method: `def complete(messages: list[dict], system: str) -> str`. Wrapping it now means swapping providers later is one file, not ten.
- 🟢 **A1.2: Add the SDK to `requirements.txt`.** (e.g., `anthropic`, `openai`, or `google-generativeai`.) Commit and tell the team to `pip install -r requirements.txt`.
- 🟢 **A1.3: Write the Concierge system prompt.** Draft in a markdown file inside `agents/prompts/concierge.md`. Should instruct the model to act as a warm, conversational travel assistant. Include: tone, what info to ask for, what languages to support (Romanian? English? both?).
- ⚠️ **DECISION A1.4: Conversation language.** The current UI is in Romanian. Decide: does the AI reply in Romanian, English, or auto-detect from the user's message? Pin the answer in this doc.

- 🟢 **A1.5: Build `agents/concierge.py`** with `generate_reply(prompt, history, preferences) -> str`. Loads the system prompt, formats history into the provider's message format, calls `llm_client.complete()`, returns the string.
- 🟢 **A1.6: Wire it into `orchestrator.py`.** Replace the stub. The orchestrator fetches `UserPreference` for the user, fetches the last N messages from the session (start with N=20), passes both to Concierge, returns the reply.
- 🟢 **A1.7: Manual testing.** Run the server, send 10 different prompts ("I want to go to Italy", "What's the weather in Paris?", "Recommend a 3-day Rome itinerary"). Note where the AI gives bad answers — iterate on the system prompt.
- 🟢 **A1.8: Error handling.** Wrap the LLM call in try/except. If the API fails, return a friendly fallback message instead of crashing. Log the error to console.

🔄 **SYNC POINT — End of Week 1:** Demo working Agent 1 to teammates. By now Branch B has hopefully shipped basic UX polish (A1.7 testing is more fun in a styled chat than the raw prototype).

Week 2 — Agent 2 (Data Architect)

- 🟢 **A2.1: Write the Data Architect system prompt.** This is the hard one. The prompt must instruct the model to return *only* JSON matching the `UserPreference` schema, with `null` for unknown fields. Iterate at least 10 times. Test on at least 15 sample conversations.
- 🟢 **A2.2: Build `agents/data_architect.py`** with `extract_preferences(history) -> dict`. Call the LLM, strip markdown code fences if present (LLMs love to wrap JSON in ```` ```json ````), `json.loads` it, validate against expected keys.
- 🟢 **A2.3: JSON validation layer.** Don't trust the LLM. Build a function that takes the parsed dict and: drops unknown keys, coerces types (`hotel_stars` must be int 1-5, `budget` must be Decimal, `travel_pace` must be one of the choices), discards invalid values.
- 🟢 **A2.4: Merge strategy.** When Agent 2 returns new preferences, you need to decide whether to overwrite existing values or only fill in nulls. **Recommended:** only overwrite if the new value is non-null AND differs from existing. Never null-out an existing value.
- 🔴 **BLOCKER A2.5:** Coordinate with Branch C before writing to `UserPreference`. They may have added an `updated_by_ai_at` timestamp field or similar. **Wait for Branch C's migration to land in main before testing this end-to-end.**
- 🟢 **A2.6: Wire into orchestrator (synchronous).** After Concierge returns its reply, call Data Architect, validate, merge, save. The user sees Concierge's reply only after both finish — slower but correct.

🔄 **SYNC POINT — End of Week 2:** All three branches should now work together. Do a full team integration test: sign up new user, chat for a few turns, check that `UserPreference` row has been populated, check that Branch C's preferences page reflects it.

Week 3 — Async + polish (stretch goals)

- 🟢 **A3.1: Make Agent 2 async** using `threading.Thread`. Wrap in try/except so a crash doesn't take down anything. Important: call `connection.close()` inside the thread before exiting, or you'll leak DB connections.
- 🟢 **A3.2: Conversation truncation.** Decide what happens when chat history exceeds the model's context window. Simplest: keep only the last 30 messages. Better: summarize older messages into a "context summary" passed in the system prompt.
- 🟢 **A3.3: Logging.** Add a simple logger that writes every LLM call (model, tokens, latency, success/fail) to a log file. Invaluable for debugging.
- 🟢 **A3.4 (Stretch): Add Amadeus API or similar** for real flight/hotel data, called from Concierge as a tool. Only attempt if everything else is done — this is a project on its own.

BRANCH B — Chat UX (`feature/chat-ux`)

Owner: *[Teammate 2 name here]* **Estimated effort:** 20–30 hours. **Key files:** `trips/templates/html/chat.html`, `resurse/css/chat.css`, possibly new `chat.js` file, `trips/views.py` (response format only).

Week 1 — Foundation polish

- 🟢 **B1.1: Audit current chat UI on mobile.** Open the app on your phone (or browser dev tools at 375px width). The sidebar at 260px will eat half the screen. Make a list of broken things.
- 🟢 **B1.2: Mobile sidebar.** Add a hamburger toggle. Sidebar slides in from the left on small screens, hidden by default. Keep the current desktop behavior at >768px width.
- 🟢 **B1.3: Markdown rendering.** Add `marked.js` via CDN. When rendering assistant messages, parse markdown to HTML. Sanitize with DOMPurify (also via CDN) — the LLM could output dangerous HTML otherwise.

- 🟢 **B1.4: Typing indicator component.** Three animated dots that appear in the messages area while waiting for a response. Pure CSS animation, no JS framework needed.
- 🟢 **B1.5: Auto-resize textarea improvements.** Current implementation works but breaks on paste of long content. Fix it. Cap height at ~200px so a giant paste doesn't push the send button off-screen.
- 🟢 **B1.6: Empty state polish.** The current empty state ("Unde vrei să călătorești?") is okay but could be better. Add 3–4 example prompt buttons ("Plan a weekend in Paris", "Find me a cheap beach trip", etc.) that auto-fill the textarea when clicked.

🔄 **SYNC POINT — End of Week 1:** Polished UI on top of Branch A's working Agent 1. Shouldn't require Branch A to do anything new — uses the existing form-POST flow.

Week 2 — Convert to AJAX

- 🔴 **BLOCKER B2.1: Coordinate with Branch A on response format.** Per Decision 0.4 above, the endpoint should return JSON. Confirm Branch A is ready for `trips/views.py` to change to a JSON response, OR add a new endpoint (`/chat/<id>/message/`) that returns JSON, keeping the old one as fallback.
- 🟢 **B2.2: Write `chat.js`** (new file in `resurse/js/`). Handles the fetch call: serializes form, POSTs to the endpoint with CSRF token, awaits JSON, appends the assistant message to the DOM, manages the typing indicator.
- 🟢 **B2.3: Update `views.py` (or add new endpoint)** to return JSON for AJAX requests. Use Django's `JsonResponse`. Distinguish AJAX from form submits via the `X-Requested-With` header or a dedicated URL.
- 🟢 **B2.4: Optimistic UI.** When user submits, immediately append their message to the DOM (don't wait for the server roundtrip). Replace with the real saved version after the response arrives.
- 🟢 **B2.5: Error states.** If the AJAX call fails, show a "couldn't send — try again?" inline error. Don't lose the user's typed message.

🔄 **SYNC POINT — End of Week 2:** Full AJAX chat working with both AI agents.

Week 3 — Stretch goals

- 🔴 **BLOCKER B3.1: Streaming responses.** Requires Branch A to expose a streaming endpoint, which requires the chosen LLM provider to support it (Claude, OpenAI yes; some others no). **Discuss with Branch A in Week 2** whether this is feasible before committing.
- 🟢 **B3.2: Server-Sent Events** on the frontend if B3.1 is feasible. Replace `fetch` with `EventSource`. Append text to the message bubble as tokens arrive.

- 🟢 **B3.3: Delete session button.** Small × icon on each sidebar item. Confirmation modal. AJAX-deletes via a new endpoint. **Coordinate with Branch A** — adding a delete endpoint to `trips/urls.py` is a `views.py` edit.

  aici ma gandeam sa fac un meniu cand dai click dreapta si sa iti apara optiunea de stergere, design-ul cu X nu prea imi place, butonul de rename il putem baga in acest meniu pe care il dai cand apesi click dreapta sa faci acelasi lucru, sa poti sa dai rename la sesiunea de chat

- 🟢 **B3.4: Rename session.** Double-click sidebar title to edit. Currently the title is just the first 30 chars of the prompt — let users override it.

BRANCH C — User Preferences (`feature/user-preferences`)

Owner: *[Teammate 3 name here]* **Estimated effort:** 15–25 hours (the shortest branch — this person should help with Branch A in Week 3). **Key files:** `users/views.py`, `users/forms.py` (new), `users/templates/html/`, `users/models.py`, `resurse/css/`.

Week 1 — Preferences page

- 🟢 **C1.1: Build `PreferenceForm`** as a Django ModelForm bound to `UserPreference`. Style fields with the same look as your existing signup/login forms (consistency matters).
- 🟢 **C1.2: Create `preferences_view`** in `users/views.py`. GET shows the form pre-filled, POST validates and saves. Use `@login_required`. Use `UserPreference.objects.get_or_create(user=request.user)` to handle first-time users gracefully.
- 🟢 **C1.3: URL + template.** Add `path('preferences/', ...)` to `users/urls.py`. Create `users/templates/html/preferences.html` matching your existing visual style (the orange Trippi brand, glass-card aesthetic).
- 🟢 **C1.4: Sidebar link.** Add a "Preferences" link in `chat.html` sidebar, above the logout button. **This is a 2-line edit to a file Branch B owns — communicate before pushing.**
- 🟢 **C1.5: Field-level UX.** Hotel stars should be a 1–5 visual rating (5 clickable star icons), not a number input. Travel pace should be radio buttons or a segmented control. Interests should be a tag input or a comma-separated textarea with placeholder examples.

🔄 **SYNC POINT — End of Week 1:** Working preferences page. At this point Branch A doesn't need it yet (Concierge just reads whatever is in the DB, which is empty/default — that's fine).

Week 2 — Onboarding flow + AI integration awareness

- 🟢 **C2.1: Schema decision: add `updated_by_ai_at` field.** When Agent 2 writes to a field, we want to track which fields came from AI vs user. Add a JSONField like `ai_updated_fields = JSONField(default=dict)` storing `{"hotel_stars": "2026-05-19T...", ...}` per AI-updated field. Generate the migration.
- 🔴 **BLOCKER C2.2: Communicate this schema change to Branch A immediately.** They need to update their merge logic in A2.4 to populate `ai_updated_fields` when they write to the model. **Don't merge C2.1 without coordinating.**
- 🟢 **C2.3: "Inferred from chat" badges.** In the template, next to each field, show a small "✨ inferred from your chats — click to override" tag if `ai_updated_fields` contains that field. Removes the tag when the user manually edits.
- 🟢 **C2.4: Onboarding redirect.** After signup, instead of redirecting to `/chat/`, redirect to `/preferences/?onboarding=1`. Template shows a "Tell us about yourself (or skip)" version with a "Skip — I'll tell the AI as we go" button that just sets a session flag and redirects to chat.
- 🟢 **C2.5: Welcome state.** First time the user lands on chat after onboarding, show a customized greeting message in the empty state based on what they filled in.

🔄 **SYNC POINT — End of Week 2:** Preferences + onboarding flow + AI agents all integrated. Do the team test from Branch A's Week 2 sync point.

Week 3 — Trip history + help Branch A

- 🟢 **C3.1: Trip history page.** List view at `/trips/` showing all the user's `ChatSession`s, ordered newest first, with title, date, message count. Each is a link to that session.
- 🟢 **C3.2: Session preview cards.** Show the first user message as a preview snippet on each card. Looks much more polished than just titles.
- 🤝 **C3.3: PAIR with Branch A on async Agent 2 (A3.1).** This is the single hardest piece of the project — having a second pair of eyes during the threading/connection-handling implementation will save days. Both names go on the commits.
- 🟢 **C3.4 (Stretch): Account settings page.** Change email, change password, delete account. Standard stuff but rounds out the product.

Cross-Cutting Concerns (Everyone's Job)

- 🟢 **DAILY: 10-minute standup.** What I did yesterday, what I'm doing today, what's blocking me. Use Discord/Slack/WhatsApp voice — async text works too but is slower.
- 🟢 **DAILY: Rebase on main.** Before starting work each day: `git fetch origin && git rebase origin/main`. Catches conflicts when they're small.
- 🟢 **PR REVIEW.** Every PR needs at least one reviewer from the other two. Even a skim. Catches bugs and spreads knowledge.
- ⚠️ **DECISION: Test data.** Decide whether to create a `manage.py loaddata` fixture with a test user + sample chat so all three of you can develop against the same baseline. Recommended — takes 30 minutes and saves hours.
- ⚠️ **DECISION: Where do AI API costs live?** If you're using a paid API, decide whose key + whose billing. Or share a single team key in `.env` (never commit it).

Risk Register — Things That Will Go Wrong (and what to do)

Risk	Likelihood	Mitigation
Agent 2 produces malformed JSON	High	A2.3 validation layer + try/except. Log failures, never crash.
Merge conflict in `trips/views.py`	High	Pre-refactor in Task 0.5 minimizes this. Communicate before editing.
LLM costs spiral	Medium	Set a hard monthly budget on the provider dashboard. Log token usage (A3.3).
One teammate falls behind	Medium	Weekly sync points catch this. Branch A owner is most at risk — that's why C3.3 pairs them up.
Romanian/English language inconsistency	Medium	Resolved by Decision A1.4. Lock it in early.

Database migration conflicts	Low (with discipline)	Only Branch C touches `UserPreference` schema. Only Branch A touches `agents/` migrations.

Definition of Done — When Is the Project "Shipped"?

Agree on this together in Week 0 and pin it here. Suggested:

1. A new user can sign up, complete onboarding (or skip it), and immediately chat with the AI.
2. The AI responds in under 5 seconds for the first message (under 10 with sync Agent 2).
3. After 3–4 chat turns, the user's `UserPreference` row has been meaningfully populated.
4. The user can visit `/preferences/` and see what the AI inferred about them, and override any field.
5. The user can revisit any past chat session from the sidebar or trip history.
6. The app works on a phone screen.
7. No console errors in the browser during a normal flow.
8. `README.md` explains how to run the project locally (env vars, migrations, runserver).